Pole

- uspořádaná, měnitelná kolekce prvků s duplicitami s pevnou délkou
- nízkoúrovňová datová struktura
- Přístup na index $\Theta(1)$
- Zvětšování pole
  - ideální je zvětšovat na dvojnásobek

Seznam implementovaný polem (klasický python List)

- uspořádaná, měnitelná kolekce prvků s duplicitami
- umožňuje přidání
- Podporované operace
  - vložení na jakoukoliv pozici
  - vybrání i odebrání prvku z jakékoliv pozice
  - procházení seznamu
- Vložení prvku do seznamu velikosti n na pozici k
  - pokud $n + 1 > 2^k$ ($k = \lceil \log2(n) \rceil$), pole zvětšíme
  - posun všech prvků: $arr[i + 1] = arr[i]$ pro $i = n - 2 ... k$
  - přepsání prvku na pozici k
  - **složitost $\Theta(n)$**
- Odebrání prvku ze seznamu velikosti n z pozice k
  - posun všech prvků: $arr[i] = arr[i + 1]$ pro $i = k ... n - 2$
  - kontrola velikosti pole: pokud $n - 1 < 2^{k-1}$, zmenšíme
- Složitosti
  -

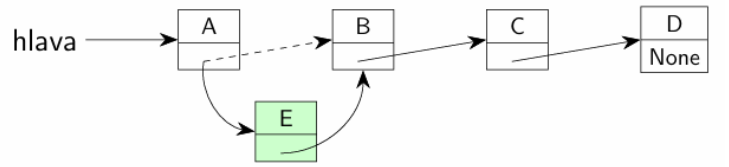
Operace	Vysvětlení	Složitost
list[index]	Přístup k prvku na konkrétní pozici	O(1)
list.append(x)	Přidání prvku na konec seznamu	O(1)
list.pop()	Odebrání a vrácení posledního prvku seznamu	O(1)
list.pop(i)	Odebrání a vrácení prvku na i-té pozici	O(n)
list.insert(i, x)	Vložení prvku na konkrétní pozici	O(n)
del list[i]	Odebrání prvku na konkrétní pozici	O(n)
for x in list	Projití seznamu	O(n)
len(list)	Získání délky seznamu	O(1)
max(list)	Získání největšího prvku seznamu	O(n)
min(list)	Získání nejmenšího prvku seznamu	O(n)
x in list	Zkontrolování, zda je prvek v seznamu	O(n)
list1 + list2	Sloučení dvou seznamů	O(k)
list * k	Opakování seznamu k-krát	O(nk)
list.remove(x)	Odebrání prvního výskytu prvku	O(n)

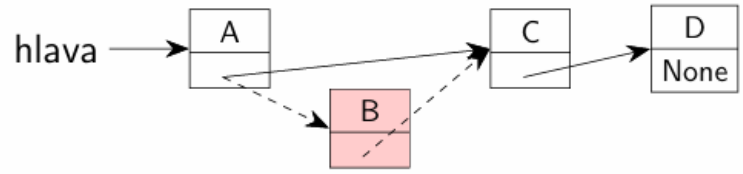
Zásobník implementovaný polem

- Uspořádaná, měnitelná kolekce prvků s duplicitami.
- Podporované operace
  - přidání prvku na konec
  - vybrání prvku z konce
  - odebrání prvku z konce
- Implementace podobná jako Seznam
  - ale jak pracujeme jen s koncem, nemusíme prvky posouvat
  - složitost všech operací díky tomu **$\Theta(1)$**

Spojový seznam

- Řeší problémy pole a jeho implementací
  - přidání prvku je někdy rychlé a někdy pomalé
    - občas potřebujeme záruku, že operace proběhne v nějakém "krátkém" čase
  - pole může zabírat místo v paměti, které aktuálně není potřeba
- Vhodné užití
  - když chceme alokovat velké množství malých kousků paměti
  - nemůžeme dát každému kousku vlastní identifikátor
- Implementace – třída s 2 atributy
  - data
  - odkaz na další prvek
  -

  - ```
    class Node:
        def __init__(self, data):
            self.data = data # Data uzlu
            self.next = None # Ukazatel na další uzel
    ```
- Přidání prvku
  - 

  - ```
    class LinkedList:
        ...
        # Vložení nového uzlu za daný uzel v seznamu
        def insert_after_node(self, data: int, node: Node) -> None:
            new_node = Node(data) # Vytvoříme nový uzel s daty
            new_node.next = node.next # Nastavíme ukazatel nového uzlu na další uzel daného uzlu
            node.next = new_node # Nastavíme ukazatel daného uzlu na nový uzel
    ```
- Odebrání prvku
  - prvek „přemostíme“
  - 

  - ```
    ...
        # Smazání uzlu za daným uzlem v seznamu
        def delete_after_node(self, node: Node) -> None:
          node.next = node.next.next # Nastavíme další ukazatel daného uzlu na další uzel dalšího uzlu
    ```
    ```

Zásobník implementovaný spojovým seznamem

- Uspořádaná, měnitelná kolekce prvků s duplicitami
- Podporované operace:
  - přidání prvku na konec
  - vybrání prvku z konce
  - odebrání prvku z konce
- Implementace:
  - strukturu si virtuálně otočíme a konec si dáme na začátek
  - prvky přidáváme a odebíráme ze začátku (měníme ukazatel hlavy)
  - hlídáme si jen konec zásobníku
  - **složitost Θ(1)**

Fronta implementovaná spojovým seznamem

- Uspořádaná, měnitelná kolekce prvků s duplicitami
- Podporované operace:
  - přidání prvku na konec
  - vybrání prvku na začátku
  - odebrání prvku ze začátku
- Implementace:
  - strukturu si virtuálně otočíme a konec si dáme na začátek
  - prvky přidáváme na konec a odebíráme ze začátku (měníme ukazatel hlavy)
  - hlídáme si jen konec fronty
  - **složitost Θ(1)**

implementace polem VS spojovým seznamem

- Implementace polem umožňují vybrání prvků na pozici s konstantní složitostí.
- Implementace spojovým seznamem umožňují vložení a odebrání prvku na aktuální pozici konstantní složitostí.
- Spojové seznamy je možné spojit s konstantní složitostí.
- Implementace polem nezatěžuje tolik systém správy paměti.

Systém hromadné obsluhy

- Umožňuje efektivně modelovat a analyzovat opakující se operace a události.
- Simulace SHO nabízí flexibilní nástroj pro pozorování dopadů změn parametrů na systém.
- Základem jsou fronty, aktivní prvky simulující reálné objekty a princip modelování času.